

Software Architecture and Design Specification

Project: Railway Ticket Reservation System

Version: 1.0

Authors: Monisha Agarwal, Lakshmi Pranathi Tummalapenta, Mohammed Farhan Ahmed, Nandita N Patil (Group 15)

Instructor: Prof. Niveditha Reddy

Date: 06-10-2026

Status: Final

Related Documents: Railway Ticket Reservation System SRS v1.0; Software Test Plan v1.0

Revision History

Version	Date	Author	Change Summary
1.0	06-10-2026	Group 15	Initial architecture and design specification derived from SRS v1.0

Approvals

Role	Name	Signature / Date
Instructor	Prof. Niveditha Reddy	
Course Coordinator		

1. Introduction

1.1 Purpose

This document specifies the architecture and design of the Railway Ticket Reservation System. It translates the requirements in the SRS (v1.0) into a modular structure, defines how the modules interact, and records the design decisions taken along with their reasons, so that the system can be implemented and tested consistently.

1.2 Scope

Covers every function in the SRS: train listing and search (by source/destination and by train number), seat availability, ticket booking with passenger validation, seat allocation, PNR generation and fare calculation, booking lookup, ticket cancellation with refund calculation, persistent booking records, and administrator functions (login, add/modify/delete train, view trains and bookings). The system is a standalone, console-based C++ application using local text files; payment gateways, real railway servers and network communication are out of scope.

1.3 Audience

Project developers (primary), testers writing and executing the Software Test Plan, instructors and evaluators.

1.4 Definitions

Term	Meaning
SRS	Software Requirements Specification
SAD	Software Architecture and Design Specification (this document)
STP	Software Test Plan
RTM	Requirements Traceability Matrix (SRS section 8)
PNR	Passenger Name Record, the unique booking identifier
DS-xx	Design specification identifier used in the RTM (for example DS-Booking03)
ADR	Architecture Decision Record
UML	Unified Modeling Language
STRIDE	Threat classification: Spoofing, Tampering, Repudiation, Information disclosure, Denial of service, Elevation of privilege

2. Document Overview

2.1 How to use this document

Section 3 describes the architecture (structure, rationale, security). Section 4 describes the detailed design (data, interactions, module interfaces, error handling, user interface). Developers should read 3.4 and 4.3 before coding a module; testers should use 3.8 to confirm that every requirement and test case maps to a module.

2.2 Related documents

- Railway Ticket Reservation System SRS, Version 1.0 (requirements, RTM, use-case diagrams)
- Railway Ticket Reservation System Software Test Plan, Version 1.0

3. Architecture

3.1 Goals & Constraints

Goals

- Correctness of seat and booking data: seat counts stay consistent after every booking and cancellation (RT-NF002).
- Persistence: confirmed and cancelled bookings survive an application restart (RT-F020, RT-NF004).
- Search results within 2 seconds for up to 10,000 train records (RT-NF001).
- Robustness: invalid input never crashes the program or modifies a record (PRJ-SR001, PRJ-SR002).

- Modularity: train, passenger, ticket and reservation logic can be changed independently (RT-NF005).
- Simplicity: usable by someone with basic computer knowledge through a menu-driven console (RT-NF003).

Constraints

- Console-based C++ application built with a standard compiler (g++/GCC) on Windows or Linux.
- Local text files only; no database server, network, payment gateway or real-time railway API.
- Train and fare data are simulated for academic purposes.

3.2 Stakeholders & Concerns

Stakeholder	Concerns	Addressed in
Passenger	Easy search and booking; correct PNR, seat and fare; reliable cancellation and refund	3.4, 4.2, 4.5
Administrator	Controlled access; accurate train and booking data	3.9, 4.3
Developers	Clear module boundaries; simple data formats	3.4, 3.5, 4.1, 4.3
Testers	Traceability from requirement to module to test; testable interfaces	3.8, 4.3, 4.4
Instructor / Evaluators	Completeness against the SRS; justified design decisions	3.5, 3.8

3.3 Component (UML) Diagram

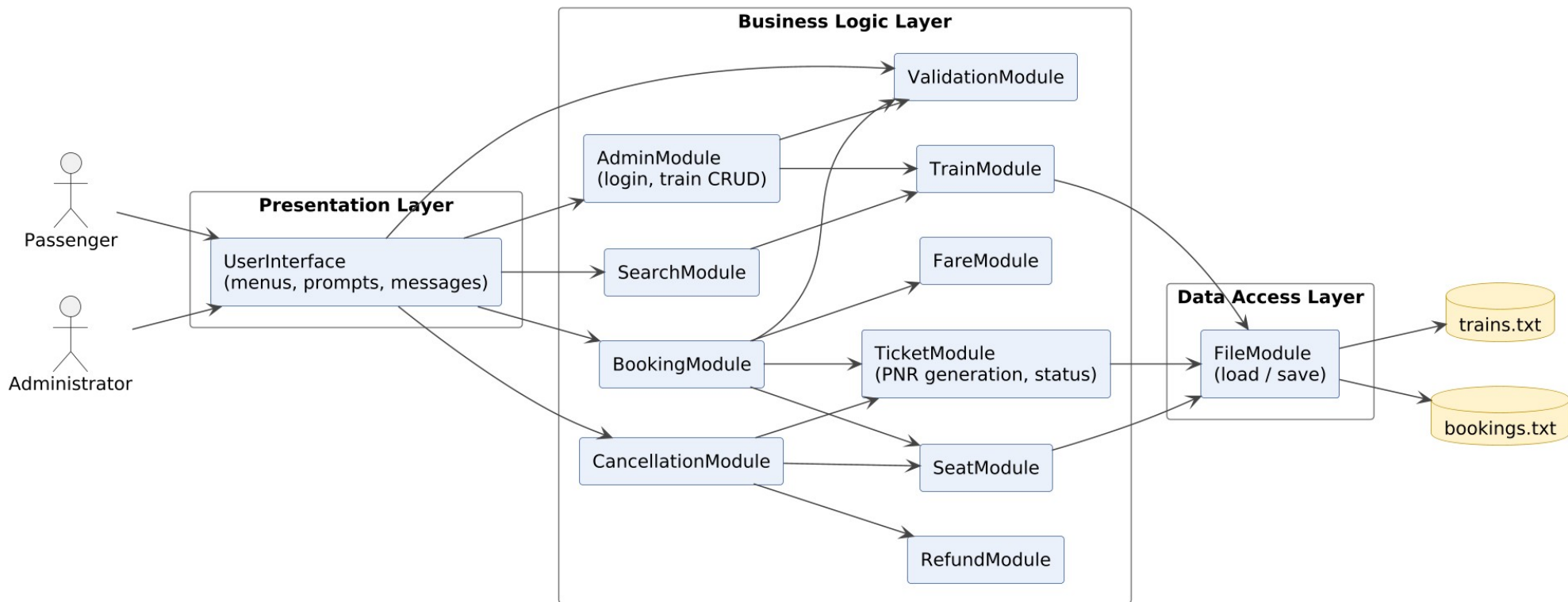


Figure 1 - UML component diagram showing the three layers, the modules and the data files

3.4 Component Descriptions

Component	Responsibility	Depends on
UserInterface	Menus, prompts, confirmations and messages for both roles; reads console input and passes it to the modules; contains no business rules.	ValidationModule, SearchModule, BookingModule, CancellationModule, AdminModule
ValidationModule	Validates passenger details, train numbers, PNR format and menu input; rejects bad input without side effects.	(none)
TrainModule	Holds the in-memory train collection; list, find by number, add, modify, delete. Train numbers must be unique; modifications cannot reduce total capacity below confirmed bookings.	FileModule
SearchModule	Search by source and destination or by train number; returns matching trains.	TrainModule
SeatModule	Seat availability per train, assigns the lowest free seat number, releases a seat on cancellation.	FileModule
FareModule	Returns the predefined fare of the selected train.	TrainModule
TicketModule	Generates unique PNRs, stores and finds bookings, sets booking status.	FileModule
BookingModule	Orchestrates the booking flow: validate, check seats, allocate seat, fare, PNR, save, confirm.	ValidationModule, SeatModule, FareModule, TicketModule
CancellationModule	Orchestrates cancellation: look up PNR, block repeat cancellation, set CANCELLED, release seat, refund.	TicketModule, SeatModule, RefundModule
RefundModule	Computes the refund amount using the 10% cancellation charge defined in the SRS.	(none)
AdminModule	Administrator login; add, modify, delete train; view all trains and bookings. Every operation checks the authenticated flag. Train deletion is rejected when confirmed bookings exist.	TrainModule, TicketModule, ValidationModule
FileModule	Only component that reads or writes files: load at start-up, save after every change.	trains.txt, bookings.txt, admin.txt

3.5 Chosen Architecture Pattern and Rationale

Pattern: three-layer architecture (Presentation, Business Logic, Data Access) implemented as separate C++ classes with a header and source file per module. The Presentation Layer communicates with the Business Logic Layer. Business Logic modules may communicate with other Business Logic modules, while only FileModule directly accesses persistent storage.

Why: the SRS requires logically independent components (RT-NF005). Isolating console I/O from business rules lets the booking and cancellation logic be tested without typing at a menu, and isolating file access means the storage format can change without touching business rules. It is also the smallest structure that fits a single-user, single-process program.

Alternatives rejected

- **Microservices / client-server:** no network is allowed (SRS 3.4) and there is one user at a time, so the extra complexity buys nothing.

- **Single-file procedural program:** fastest to write, but breaks RT-NF005 and makes unit testing of individual rules hard.
- **Embedded database (for example SQLite):** SRS 2.5 fixes local file storage, and the data is small enough for text files.

Key design decisions

ID	Decision	Reason
ADR-1	Layered architecture with one class per module	Meets RT-NF005; testable in isolation
ADR-2	Pipe-delimited text files, one record per line	Human-readable for testers; no library needed; satisfies the local-file constraint
ADR-3	Load all records into memory at start-up, save to file after every successful change	Fast search (RT-NF001); a crash loses at most the operation in progress
ADR-4	Save by writing a temporary file and then replacing the original	Prevents a half-written file from corrupting stored data
ADR-5	PNR = highest stored PNR + 1, starting at 1000000001 and kept as a 10-digit number. If 9999999999 is reached, no new booking is created.	Unique across restarts without a separate counter file (PRJ-SR003)
ADR-6	Available-seat count is recomputed from CONFIRMED bookings on load rather than trusted from the file	Avoids drift between seat counts and bookings (RT-NF002)
ADR-7	Seat number = lowest unoccupied seat from 1 to total seats	Deterministic, easy to verify in TC-BOOK-04 and TC-CAN-04

3.6 Technology Stack & Data Stores

Item	Choice
Language / standard	C++ (C++17), standard library only (vector, map, string, fstream)
Compiler / platforms	g++ / GCC on Windows or Linux
User interface	Menu-driven console
Persistent storage	Three local text files (below); no database server
Design tooling	PlantUML for diagrams

Data stores (one record per line, fields separated by |; the | character is rejected by input validation so it cannot appear inside a field):

File	Record layout	Notes
trains.txt	trainNo name source destination departure arrival fare totalSeats	Simulated train data; fare is the predefined ticket fare; availableSeats is derived at runtime and is not stored.

File	Record layout	Notes
bookings.txt	pnr trainNo name age gender contact seatNo fare bookingDate status refund	status is CONFIRMED or CANCELLED; refund is 0 until cancelled
admin.txt	username password	Administrator credentials. Stored in plain text for this offline academic prototype; see Security Architecture and A-1.

3.7 Risks & Mitigations

Risk	Mitigation
Data file missing or contains a malformed line	FileModule creates an empty file if absent; a malformed line is skipped and counted, and the count is shown at start-up so the problem is visible
Crash or power loss while saving corrupts a file	Write to a temporary file, then replace the original (ADR-4)
Seat count and bookings disagree	Recompute availability from CONFIRMED bookings on load (ADR-6); seat release and status change happen in one operation followed by one save
Duplicate PNR	PNR derived from the highest stored PNR (ADR-5); generation checks the booking collection before returning
Search slows down with 10,000 trains	Records are held in memory and scanned linearly, which is well within 2 seconds at this size; a train-number index (map) is used for lookups by number
Invalid input crashes the program	All input is read as a line of text and validated before conversion; no unchecked cin >> into numeric types
Plain-text administrator password in admin.txt	Suitable only for the offline academic prototype; the password is not exposed in the user interface, login is limited to three failed attempts, and password hashing is identified as a future enhancement.

3.8 Traceability to Requirements

Every SRS requirement maps to a design specification ID (as used in the SRS RTM), the module that implements it, and its test case. Requirement IDs follow the format used in the SRS requirement tables and the Test Plan.

Req ID	Design Spec	Module(s)	Test Case
RT-F001	DS-Train-01	TrainModule, UserInterface	TC-TRAIN-01
RT-F002	DS-Train-02	SearchModule	TC-TRAIN-02

Req ID	Design Spec	Module(s)	Test Case
RT-F003	DS-Train-03	SearchModule	TC-TRAIN-03
RT-F004	DS-Train-04	SeatModule	TC-TRAIN-04
RT-F005	DS-Booking01	BookingModule, UserInterface	TC-BOOK-01
RT-F006	DS-Booking02	ValidationModule	TC-BOOK-02
RT-F007	DS-Booking03	SeatModule, BookingModule	TC-BOOK-03
RT-F008	DS-Booking04	SeatModule	TC-BOOK-04
RT-F009	DS-Booking05	TicketModule	TC-BOOK-05
RT-F010	DS-Booking06	FareModule	TC-BOOK-06
RT-F011	DS-Booking07	BookingModule, UserInterface	TC-BOOK-07
RT-F012	DS-Cancel-01	TicketModule, CancellationModule	TC-CAN-01
RT-F013	DS-Cancel-02	CancellationModule, UserInterface	TC-CAN-02
RT-F014	DS-Cancel-03	CancellationModule	TC-CAN-03
RT-F015	DS-Cancel-04	SeatModule	TC-CAN-04
RT-F016	DS-Cancel-05	TicketModule	TC-CAN-05
RT-F017	DS-Cancel-06	RefundModule	TC-CAN-06
RT-F018	DS-BookingMgmt-01	TicketModule, UserInterface	TC-MGT-01
RT-F019	DS-BookingMgmt-02	CancellationModule	TC-MGT-02
RT-F020	DS-BookingMgmt-03	FileModule	TC-MGT-03
RT-F021	DS-Admin-01	AdminModule, TrainModule	TC-ADMIN-01
RT-F022	DS-Admin-02	AdminModule, TrainModule	TC-ADMIN-02
RT-F023	DS-Admin-03	AdminModule	TC-ADMIN-03
RT-F024	DS-Admin-04	AdminModule	TC-ADMIN-04
RT-F025	DS-Admin-05	AdminModule, TrainModule	TC-ADMIN-05
RT-NF001	DS-Perf-01	SearchModule, TrainModule	TC-NFR-01
RT-NF002	DS-Reliability01	SeatModule	TC-NFR-02
RT-NF003	DS-UX-01	UserInterface	TC-NFR-03
RT-NF004	DS-Storage-01	FileModule	TC-NFR-04
RT-NF005	DS-Architecture01	All (layered structure)	TC-NFR-05
PRJ-SR001	DS-Sec-01	ValidationModule	TC-SEC-01
PRJ-SR002	DS-Sec-02	ValidationModule	TC-SEC-02
PRJ-SR003	DS-Sec-03	TicketModule	TC-SEC-03
PRJ-SR004	DS-Sec-04	SeatModule	TC-SEC-04
PRJ-SR005	DS-Sec-05	AdminModule	TC-SEC-05

3.9 Security Architecture

The system is offline and single-user, so the main risks are bad input, accidental corruption of data files and unauthorised use of administrator functions. The STRIDE analysis below is scoped to that.

Threat	Where it applies	Mitigation
Spoofing	Someone uses administrator functions	AdminModule requires login against admin.txt; the menu is unreachable until authenticated; 3 failed attempts return the user to the main menu (PRJ-SR005)
Tampering	Invalid input or edited files change train or booking data	All input validated before use; invalid train numbers and PNRs are rejected with no record modified (PRJ-SR001, PRJ-SR002); malformed file lines are skipped at load; atomic save (ADR-4)
Repudiation	A user denies a booking or cancellation	Low risk for a local academic tool; the PNR and status stored in bookings.txt act as the record
Information disclosure	Passenger details shown to the wrong person	Details are displayed only after a PNR lookup or in the authenticated admin view; error messages never echo contact numbers
Denial of service	Repeated bad input or oversized input	Input length limits; bad input re-prompts without changing state; booking refused when seats are zero (PRJ-SR004)
Elevation of privilege	A passenger reaches admin operations	Role split at the main menu; each AdminModule function re-checks the authenticated flag instead of relying on the menu alone

4. Design

4.1 Design Overview

The program starts by asking FileModule to load trains.txt, bookings.txt and admin.txt into memory, recomputes seat availability, and then runs the main menu loop until the user exits. Each successful change (booking, cancellation, train add/modify/delete) is saved immediately.

Core data types

Type	Attributes	Notes
Train	trainNo, name, source, destination, departure, arrival, fare, totalSeats, availableSeats	availableSeats is a derived runtime field and is not persisted in trains.txt; it is recomputed on load (ADR-6)
Passenger	name, age, gender, contact	Embedded in Booking
Booking	pnr, trainNo, passenger, seatNo, fare, bookingDate, status, refund	status is CONFIRMED or CANCELLED

Collections: vector<Train> plus a map<int,size_t> from train number to position;
vector<Booking> plus a map<string,size_t> from PNR to position.

4.2 UML Sequence Diagrams

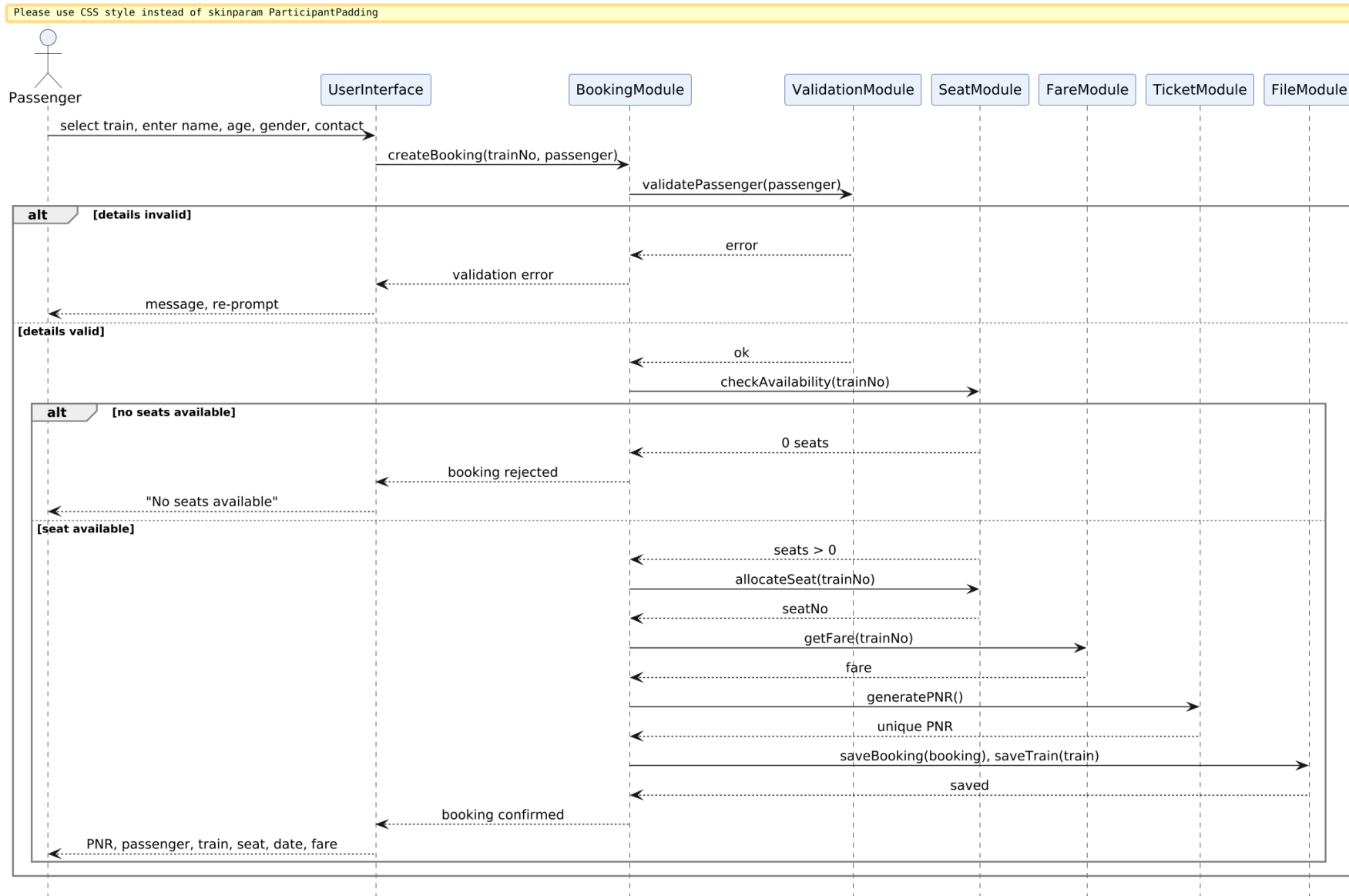


Figure 2 - Sequence 1: Book Ticket (RT-F005 to RT-F011)

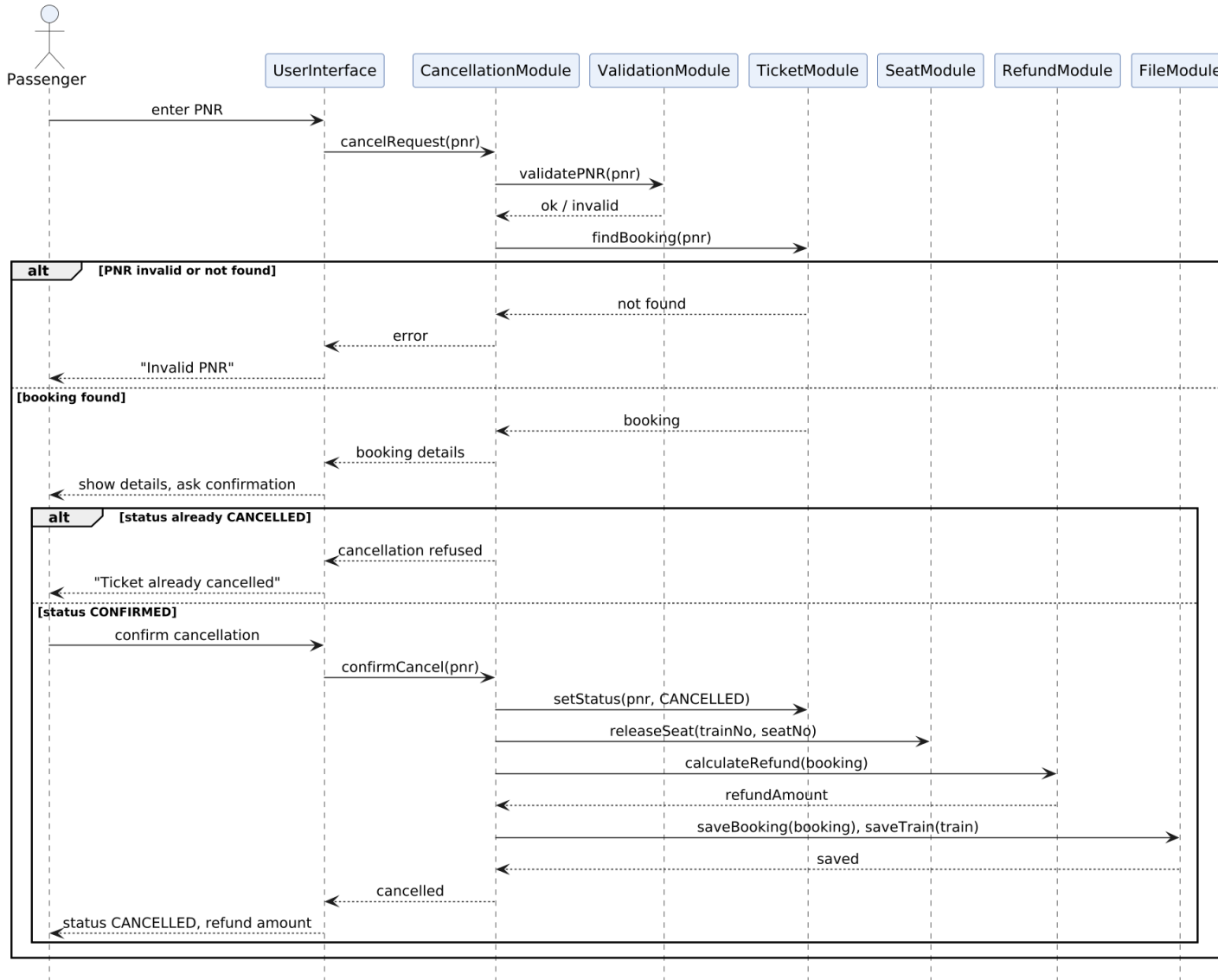


Figure 3 - Sequence 2: Cancel Ticket (RT-F012 to RT-F019)

Walk-through of the two flows. In Sequence 1, BookingModule validates the passenger, checks and allocates a seat, takes the fare, generates the PNR, saves, and returns the confirmation; a validation failure or zero seats ends the flow with a message and no data change. In Sequence 2, CancellationModule finds the booking by PNR, refuses if it is already CANCELLED, and only after user confirmation sets the status, releases the seat, computes the refund and saves.

4.3 Module Interfaces

As a single-process console program, the interfaces are C++ class methods rather than network endpoints. The tables below define the public interfaces of the main components. Result codes are the Status values in 4.4.

BookingModule

Method	Inputs	Output	Errors
createBooking	int trainNo, Passenger p	Status, Booking (PNR, seat, fare, date) on success	INVALID_INPUT, TRAIN_NOT_FOUND, NO_SEATS, FILE_ERROR

Pre: train exists, passenger passes validation. Post: on OK one CONFIRMED booking exists, available seats of the train reduced by one, data saved. On any error nothing is modified.

CancellationModule

Method	Inputs	Output	Errors
findForCancel	string pnr	Status, Booking details	INVALID_PNR, PNR_NOT_FOUND, ALREADY_CANCELLED
confirmCancel	string pnr	Status, refund amount	INVALID_PNR, PNR_NOT_FOUND, ALREADY_CANCELLED, FILE_ERROR

Post (OK): status is CANCELLED, seat released, refund stored, data saved. A booking already CANCELLED is never changed again (RT-F019).

SeatModule

Method	Inputs	Output	Errors
availableCount	int trainNo	int seats free	TRAIN_NOT_FOUND
allocateSeat	int trainNo	Status, int seatNo (lowest free)	TRAIN_NOT_FOUND, NO_SEATS
releaseSeat	int trainNo, int seatNo	Status	TRAIN_NOT_FOUND, INVALID_SEAT

FileModule

Method	Inputs	Output	Errors
loadAll	(none)	LoadReport (records loaded, lines skipped)	FILE_ERROR
saveTrains	vector<Train>	Status	FILE_ERROR
saveBookings	vector<Booking>	Status	FILE_ERROR

Other interfaces in brief

- ValidationModule: validatePassenger(p), validateTrainNo(s), validatePNR(s), readMenuChoice(min,max); each returns a Status and never throws on bad input.
- SearchModule: bySourceDest(src,dst) and byTrainNo(n) return the matching trains.
- TicketModule: generatePNR(), find(pnr), setStatus(pnr,status).
- AdminModule: login(user,pass), addTrain, modifyTrain, deleteTrain, viewTrains, viewBookings; all except login return NOT_AUTHENTICATED unless login succeeded.
- RefundModule: calculateRefund(fare) returns 90% of the fare, applying the 10% cancellation charge defined in the SRS.

4.4 Error Handling, Logging & Monitoring

Status codes. Every module method returns a Status value instead of throwing: OK, INVALID_INPUT, INVALID_PNR, PNR_NOT_FOUND, TRAIN_NOT_FOUND, NO_SEATS, ALREADY_CANCELLED, NOT_AUTHENTICATED, DUPLICATE_TRAIN_NO, TRAIN_HAS_ACTIVE_BOOKINGS, INVALID_CAPACITY, PNR_LIMIT_REACHED, FILE_ERROR. UserInterface maps each code to one fixed, plain-language message.

- **Invalid input:** the message says what was wrong (for example "Age must be between 1 and 120"), the user is re-prompted, and no record is changed.
- **File problems:** a missing file is created empty; a malformed record means an incorrect field count or invalid field value and is skipped and counted; if a save fails the in-memory change is rolled back and the user is told the operation did not complete.
- **No sensitive data in messages or logs:** contact numbers, passwords and full passenger details are never written to error output or logs.

Logging and monitoring. There is no background service to monitor. At start-up the program prints how many train and booking records were loaded and how many lines were skipped. If a debug log is added during development, it records only event type and PNR or train number. Testers can compare these counts with the data files.

4.5 UX Design

Simple numbered menus with short prompts, a confirmation step (y/n) before any cancellation, and the same wording for the same error everywhere.

MAIN MENU	PASSENGER MENU	ADMIN MENU (after login)
1. Passenger	1. View all trains	1. Add train
2. Administrator	2. Search by source/dest	2. Modify train
3. Exit	3. Search by train number	3. Delete train
	4. Check seat availability	4. View all trains
	5. Book ticket	5. View all bookings
	6. View booking (PNR)	6. Logout
	7. Cancel ticket	
	8. Back	

Input rules enforced by ValidationModule (defined here because the SRS says "valid" without specifying):

Field	Rule
Name	2 to 50 characters; letters and spaces only
Age	Whole number from 1 to 120
Gender	M, F or O (case-insensitive)
Contact number	Exactly 10 digits
Train number	Digits only; must match an existing train
PNR	10 digits; must match an existing booking
Any text field	The character is not allowed (it is the file field separator)

4.6 Open Issues & Next Steps

The following behaviors are normative implementation rules derived from the clarified SRS requirements:

- Duplicate train numbers are rejected with DUPLICATE_TRAIN_NO.
- A train cannot be deleted while it has one or more CONFIRMED bookings; deletion returns TRAIN_HAS_ACTIVE_BOOKINGS.
- Train capacity cannot be reduced below the number of CONFIRMED bookings; otherwise INVALID_CAPACITY is returned.
- Existing bookings retain the fare stored at booking time; later train-fare changes affect only future bookings.
- The booking date is the system date at booking creation; journey dates are outside Version 1.0 scope.
- The cancellation refund is 90% of the stored booking fare (10% cancellation charge).
- The first PNR is 1000000001; PNR generation stops at 9999999999 rather than producing an invalid value.

Design assumptions remaining after the SRS clarifications (to be confirmed by the group):

ID	Assumption	Affects
A-1	Administrator credentials are stored in admin.txt and compared on login. Adequate for an offline academic tool, not for production.	RT-F024, PRJ-SR005
A-2	Each train has one seat class; totalSeats is a single number per train.	RT-F004, RT-F008

Requirement IDs have been standardized across the SRS, SAD and Test Plan using the format RT-F001, RT-NF001 and PRJ-SR001.

Future enhancements: journey dates and multiple dates per train, seat classes with different fares, hashed administrator passwords, a database back end, a graphical interface, and a payment simulation.

5. Appendices

5.1 Glossary

PNR, SRS, SAD, STP, RTM, ADR, UML and STRIDE are defined in section 1.4. Booking status: CONFIRMED (seat held) or CANCELLED (seat released). Seat number: the number assigned to a

passenger after booking, unique within a train. Status codes include `DUPLICATE_TRAIN_NO`, `TRAIN_HAS_ACTIVE_BOOKINGS`, `INVALID_CAPACITY` and `PNR_LIMIT_REACHED`.

5.2 References

- Railway Ticket Reservation System SRS, Version 1.0
- Railway Ticket Reservation System Software Test Plan, Version 1.0
- ISO/IEC/IEEE 42010, Architecture description
- OWASP, Input Validation Cheat Sheet

5.3 Tools

PlantUML (diagrams), g++ (build). The PlantUML sources for the three diagrams are supplied alongside this document.